

# Analysis: Common Anagrams

## Common Anagrams: Analysis

### Small and Large dataset

We can create a boolean function  $f(i, j, k, l)$  that returns whether the substring from the  $i$ -th through the  $j$ -th characters of **A** (inclusive) is anagrammatic to the substring from the  $k$ -th through the  $l$ -th characters of **B** (inclusive).

To do this for the Small dataset, we only need to check that both substrings contain the same number of **A**s and the same number of **B**s.

To do this for the Large dataset, we can loop from  $i$  to  $j$  to get the number of occurrences of each character in **A** $[i \dots j]$ . Similarly, we also loop from  $k$  to  $l$  to get the occurrences of each character in **B** $[k \dots l]$ . We then check whether each character occurs the same number of times in the two substrings.

We want to return the number of ordered pairs  $(i, j)$  such that there exists an ordered pair  $(k, l)$  satisfying  $f(i, j, k, l)$ . Since computing a single value of  $f(i, j, k, l)$  takes  $O(L)$ , this solution runs in  $O(L^5)$  time. We can optimize this by observing that if  $j - i \neq l - k$ , then  $f(i, j, k, l)$  will certainly return false. Therefore, for each value of  $(i, j, k)$  we only need to check  $f(i, j, k, k + (j - i))$ . This removes one  $O(L)$  loop and improves the algorithm to  $O(L^4)$  time.

### Further improvements

Although  $O(L^4)$  is fast enough, we can improve the algorithm further. Instead of looping through all substrings of **B** for each substring of **A**, we can do some preprocessing. Before we even process **A**, we can loop through all substrings of **B**. For each substring, we count the occurrences of each character and store it in a hashset. Note that this can be done in  $O(L^2)$  time, since we can use the character occurrences of **B** $[i \dots j]$  to compute the character occurrences of **B** $[i+1 \dots j+1]$ .

Once the preprocess is complete, we can loop through all substrings of **A**. Similarly, for each substring, we count the occurrences of each character. We then check whether these occurrences of characters are present in our set. If they are, we can increment our answer counter. We can also do this in  $O(L^2)$  time.

Therefore, this solution runs in  $O(L^2)$  time overall.

## Analysis: Palindromic Sequence

To solve the Small dataset, we will build up the palindrome one letter at a time. Suppose we know the first  $X$  letters of the  $K$ th lexicographically least palindrome. Call this prefix of the answer  $S$ . We can guess the next letter and see how many palindromes there are with the new prefix (we will address this computation shortly). If there are more than  $K$ , then we know we need to guess a lexicographically smaller letter. Let  $P(S)$  denote the number of palindromes of Hannah's language that have  $S$  as a prefix. Then we are looking for the lexicographically largest character  $c$  such that  $P(S + c) \leq K$ . Naively, we have to make  $O(NL)$  guesses.

So how do we actually calculate the number of possible continuations? Fortunately, the bounds are Small enough that we don't need to do anything sophisticated. For every possible length  $M$ , check and see if the current prefix  $S$  is consistent with a palindrome of length  $M$ . If it is, then there are  $L^D$  possible palindromes of length  $M$ , where  $D = \max(0, \text{floor}((M+1)/2) - X)$ .  $d$  represents the number of characters that can be freely chosen (the rest are fixed to keep the string a palindrome). This can be done naively in  $O(N^2)$  time. In total, the Small can be solved in  $O(N^3L)$  time.

To solve the Large dataset, notice that when  $K \leq N$ , the answer is a string that consists of the letter  $a$  repeated  $K$  times. When  $K > N$ , it's important to notice that the answer is very close to  $N$ .

For example, consider the case when  $N = 10^5$  and  $L = 2$ . How many palindromes begin and end with 49900 letter  $as$ ? To make a rough count, let's consider only the palindromes with length exactly  $N$ . Of the 200 letters between the  $as$ , the first 100 are unconstrained, while the last 100 have to match the corresponding letter in the first 100. In total this is  $2^{100}$  different palindromes, which far exceeds that maximum bound for  $K$  ( $10^{18}$  is about  $2^{60}$ ).

The example above can be generalised to state that whenever  $K > N$ , the  $K$ th palindrome will begin and end with at least  $N - \log_L(K)$  copies of the letter  $a$ , leaving at most  $2 \cdot \log_L(K)$  letters in the middle that might not be the letter  $a$ . The algorithm in the Small dataset can be adapted to solve the Large dataset after these observations have been made.  $2 \cdot \log_L(K)$  is no more than 100, so this has a similar running time to the Small dataset.

# Analysis: Specializing Villages

## Specializing Villages: Analysis

### Small dataset

For the Small dataset, we can enumerate all plans, find the average of the distances for the villages, and count the plans with a minimal average distance.

### Large dataset

If each village can get the other food it needs from its nearest neighbor, then the average distance is clearly minimized. We will show that it is possible to construct this situation.

For each village  $v$ , let  $l(v)$  be the numerical label of  $v$ ,  $r(v)$  be the shortest of the roads that have  $v$  as an endpoint,  $d(v)$  be the distance of  $r(v)$ , and  $f(v)$  be the other village to which  $r(v)$  connects  $v$ . We first sort the villages by increasing  $d(v)$ . Since two villages might share the same  $d(v)$ , we break ties by increasing  $l(v)$ .

Then, we handle each village in sorted order. We assign  $v$  the opposite food of  $f(v)$ ; if  $f(v)$  does not have a food assigned, we can assign either food to  $v$ . How do we know this will not cause problems later in the process? Well, this situation can only happen when  $d(f(v)) \geq d(v)$ . But we know that  $r(v)$  connects  $f(v)$  and  $v$ , so  $d(f(v)) \leq d(v)$ , and thus  $d(f(v)) = d(v)$  in this case, which implies  $r(f(v)) = r(v)$  and  $l(f(v)) > l(v)$ . Since  $r(f(v)) = r(v)$ ,  $f(f(v)) = v$ , so  $f(v)$  will be assigned the opposite food of  $v$  when we get to it.

In our approach, some villages may have their food choices forced by the food choices of other villages, whereas some villages may be able to choose either food. So, the count of optimal plans is  $2^N$ , where  $N$  is the number of villages that can choose.  $N$  is equal to the size of  $\{v \mid f(f(v)) = v \text{ and } l(f(v)) > l(v)\}$ .

Roads with length 0 pose a special case. If a 0-length road connects villages  $x$  and  $y$ , any village  $v$  other than  $x$  and  $y$  with  $f(v)$  in  $\{x, y\}$  has two choices, since it can get one food through  $r(v)$  and the other from a combination of  $r(v)$  and the 0-length road. So, with a 0-length road present, the answer will be  $2^N$ , where  $N$  is the size of  $\{v \mid f(f(v)) = v \text{ and } l(f(v)) > l(v)\} \cup \{v \mid d(v) > 0 \text{ and } d(f(v)) = 0\}$ .